

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Now create a WebSocket configuration class:

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements
WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry
config) {
        config.enableSimpleBroker("/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry
registry) {
        registry.addEndpoint("/ws").setAllowedOrigins("*").
withSockJS();
    }
}
```

Next, we need to build a controller for message handling using a messaging template. For example, to get stock prices in real-time we can use ticker messaging as shown below:

```
@RestController
public class StockMarketController {

    @Autowired
    private SimpMessagingTemplate messagingTemplate;

    @MessageMapping("/updateStock")
    @SendTo("/topic/stockPrices")
    public Stock updateStockPrice(Stock stock) {
        stock.setPrice(stock.getPrice() + Math.random() *
10);
        return stock;
    }

    public void broadcastStockPrice(Stock stock) {
        messagingTemplate.convertAndSend("/topic/
stockPrices", stock);
    }
}
```

```
}
```

The object model for stock data to be used in the messaging transfer could be like this:

```
public class Stock {
    private String name;
    private double price;

    // Getters and Setters
}
```

Finally, here's the frontend WebSocket client example using JavaScript, which subscribes to a STOMP client to communicate as an interactive web application:

```
var socket = new SockJS('/ws');
var stompClient = Stomp.over(socket);
stompClient.connect({}, function (frame) {
    stompClient.subscribe('/topic/stockPrices', function
(message) {
        var stock = JSON.parse(message.body);
        console.log("Updated stock price: ", stock);
    });
});
```

Using WebSockets with STOMP in a Spring Boot application provides an efficient way to handle real-time interactions. Open source frameworks like Spring WebSocket and RabbitMQ help build scalable, event-driven applications. Whether for stock market updates, ticket booking systems, or chat applications, this approach ensures high responsiveness and an interactive user experience. **END** 🐧

Acknowledgements

The author would like to thank Ganesh Kumar Thiagarajan @ Sysoutsolutions.in, who helped with this solution and the code snippets.

References

- <https://spring.io/guides/gs/messaging-stomp-websocket>
- <https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html>
- <https://we-plus.be/sending-stomp-messages-over-a-websocket-in-spring-boot-2/>
- <https://www.baeldung.com/websockets-spring>

By: Dr Magesh Kasthuri

The author is a distinguished member of the technical staff and Chief Architect at Wipro Ltd. This article expresses his views and not that of Wipro.